Laboratorio 6 — Probando el backend

Profesor: Matías Toro
Ayudantes: Benjamín Indo, Benjamín Madrid, Camilo Huerta, Daniel Ramírez, Julián Ferreira, Sebastián Bustos, Sebastián González, Tomás Méndez, Tomás Ubilla

En este laboratorio se pondrán en práctica las técnicas de testing de backend vistas en clases.

Encontrarán la template en la sección de material docente en U-Cursos. Además, encontrarán una solución de referencia del laboratorio anterior; si prefiere seguir con su propio backend, reemplace la carpeta **`backend`**.

El template ya viene con `supertest` y el *test runner* de Node (`node:test`) configurados: sus pruebas van en `test/`, reciben el `app` que exporta `src/app.ts`, y se corren con `npm test`, que las apunta a una base de datos aparte de la de desarrollo.

Las pruebas de integración necesitan un MongoDB local. Con Docker basta correr `docker run -d -p 27017:27017 mongo:8`; si no, instale MongoDB Community.

P1.

Los tests unitarios sirven para probar la lógica de la aplicación de forma atómica, dígase una función o componente, de forma aislada. Además, se pueden escribir antes de implementar la funcionalidad, para apoyar el proceso de implementación.

Por el momento, el servidor, además de los endpoints, no tiene mucha lógica complicada. Suponga entonces que quiere implementar una función con la siguiente firma:

```
function filterPostsByWord(posts: Post[], query: string): Post[]
```

La función debe cumplir los siguientes requerimientos funcionales:

1. Debe entregar todos los elementos de `posts` que contengan `query` en su contenido.
   - **La búsqueda debe ser case insensitive**: no debe distinguir entre `"PALABRA"`, `"Palabra"` o `"palabra"`.
   - **Debe ignorar la puntuación**. Si `query` es `"bakemon"`, debe encontrarlo en el contenido `"Es Bakemon, no Pokemon"`.
   - **Solo debe coincidir con palabras completas**. Si `query` es `"sol"`, no debe coincidir con `"girasol"`.
2. Si `posts` está vacío, si `query` es el string vacío, o si ningún elemento contiene a `query`, debe entregar un arreglo vacío.

Escriba pruebas unitarias para cada requerimiento y solo después implemente la función, hasta que todas pasen. El tipo `Post` ya está definido en `src/models/posts.ts`.

P2.

Las pruebas unitarias no garantizan que los componentes funcionen en conjunto, porque su objetivo es probar las funciones de forma aislada. Para eso se escriben pruebas de integración.

Con `beforeEach`, limpie la colección y siembre al menos dos threads con comentarios, y escriba dos pruebas que verifiquen que se devuelve la cantidad correcta de publicaciones: una para `GET /api/`

`threads` (que debe devolver solo los threads) y otra para `GET /api/threads/:id` (el thread con todos sus comentarios). Cierre la conexión con `after` al terminar, o el proceso de pruebas nunca termina.

Importante: todo lo descrito acá y en las preguntas siguientes debe escribirse usando funciones asíncronas y `await`, donde corresponda.

P3.

Siga en el mismo archivo de pruebas, reutilizando el `beforeEach` de la P2. Pruebe ahora la forma de las respuestas y el manejo de casos inválidos. Escriba pruebas que verifiquen que:

1. El identificador único de cada thread y comentario devuelto se llama `id` y no `_id`, que es el nombre que usa `mongoose` por defecto. La respuesta tampoco debe traer `__v`.
2. `GET /api/threads/:id` con un id válido pero que no existe responde 404.
3. `GET /api/threads/:id` con un id mal formado responde 400.
4. Una petición a una ruta que no existe responde 404 y un cuerpo JSON con el error.

Luego, haga los cambios necesarios para que las cuatro pasen. Para la última necesitará un *middleware* nuevo; ponga atención en el orden en que registra los middlewares.

P4.

El esquema del laboratorio anterior impone tres restricciones: el contenido es obligatorio, su largo va entre 1 y 300 caracteres, y el autor —que es opcional— no puede ser uno de los nombres prohibidos. Ninguna está probada.

Escriba pruebas que verifiquen lo siguiente, tanto al crear un thread (`POST /api/threads`) como un comentario (`POST /api/threads/:id`):

1. Con contenido válido, responde 201 y se agrega exactamente una publicación.
2. Si el contenido no viene en la petición, responde 400 y no se agrega nada.
3. Si el contenido excede los 300 caracteres, responde 400 y no se agrega nada; con exactamente 300, se acepta.
4. Si el contenido es el string vacío, responde 400.
5. Si `author` no viene, la publicación se guarda con éxito. Verifique que quedó agregada, no solo el código de respuesta.
6. Si `author` es uno de los nombres prohibidos (**Huevito rey**, **Matías Toro**, **Memes es mal ramo**), responde 400 y no se agrega nada. Incluya un caso con otra capitalización, por ejemplo `"HUEVITO REY"`.

Para verificar si la publicación se agregó, consulte la base antes y después de la petición y compare las cantidades. Luego haga los cambios necesarios para que todas pasen.

P5.

Falta probar que un comentario no pueda quedar colgando. Escriba pruebas que verifiquen que:

1. `POST /api/threads/:id` con un id de thread válido pero que no existe responde 404 y no crea el comentario.

2. `POST /api/threads/:id` con un `parent` que no pertenece a ese thread responde 400 y no crea el comentario. Su `beforeEach` ya siembra dos threads con comentarios, así que puede usar un comentario del segundo como `parent` inválido para el primero.
3. `POST /api/threads/:id` con un `parent` que sí es un comentario de ese thread responde 201.

Luego haga los cambios necesarios para que las tres pasen.

P6.

Queda un endpoint sin probar, y es el que el frontend usa en cada like: el `PUT /api/posts/:id`. Escriba pruebas que verifiquen que:

1. Un PUT con un cuerpo válido responde 200 y el cambio queda guardado. Por ejemplo, subir los likes de un thread.
2. Un PUT con un id válido pero que no existe responde 404.
3. Un PUT que deja el contenido por sobre los 300 caracteres responde 400 y no modifica la publicación.
4. Un PUT que deja el contenido vacío responde 400 y no modifica la publicación.
5. Un PUT que pone un autor prohibido responde 400 y no modifica la publicación.

Las tres últimas son las mismas restricciones de la P4, pero llegando por la vía de la actualización. Verifique en cada caso qué quedó realmente guardado, no solo el código de respuesta. Luego haga los cambios necesarios para que las cinco pasen.